



Modular, Full-Stack Verification

Gregory Malecha, Hoang-Hai Dang, Paolo G. Giarrusso,
Simon Hudon, Jan-Oliver Kaiser (BlueRock Security)
David Swasey (Riverside Research)

gregory@bluerock.io



Modular, Full-Stack Verification

Gregory Malecha, Hoang-Hai Dang, Paolo G. Giarrusso,
Simon Hudon, Jan-Oliver Kaiser (BlueRock Security)
David Swasey (Riverside Research)

gregory@bluerock.io

We can *specify real, concurrent systems* in ways that *naturally align with systems design principles*.

Full verification of real systems code is tractable.

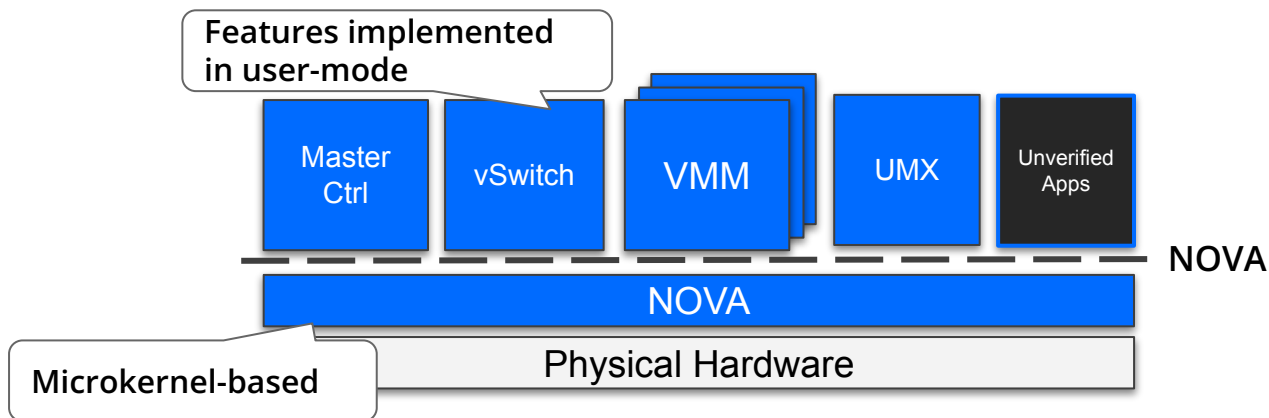
The Challenge

A Virtualization Platform



The Challenge

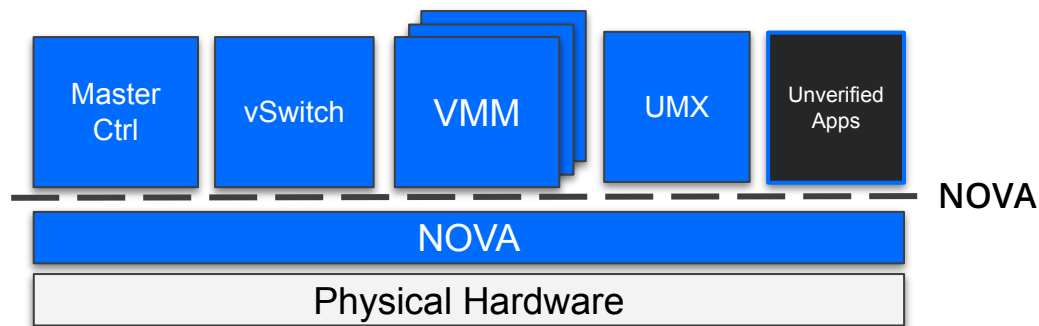
A Virtualization Platform



The Challenge

A Virtualization Platform

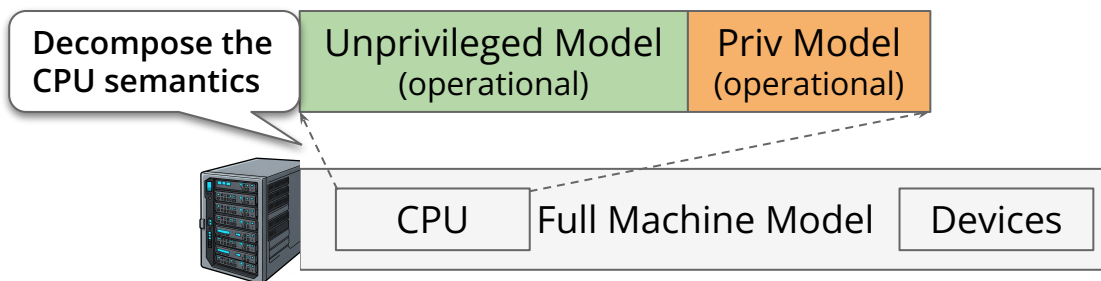
1. **Real System** – concurrency, dynamic resource sharing, real systems engineers (+mainstream PL, e.g. C++).
2. **Two-sided Specification** – single specification used by both the implementation and the clients.
3. **Modular Verification** – aligned with the architecture of the actual systems engineers.
4. **Robust** – support inter-operation with unverified code.



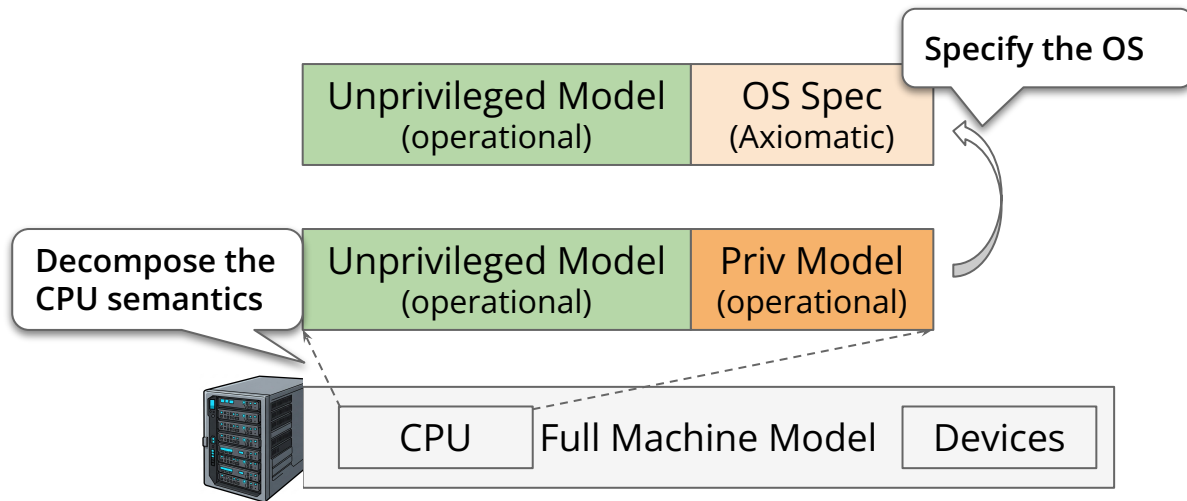
The Full-Stack Proof



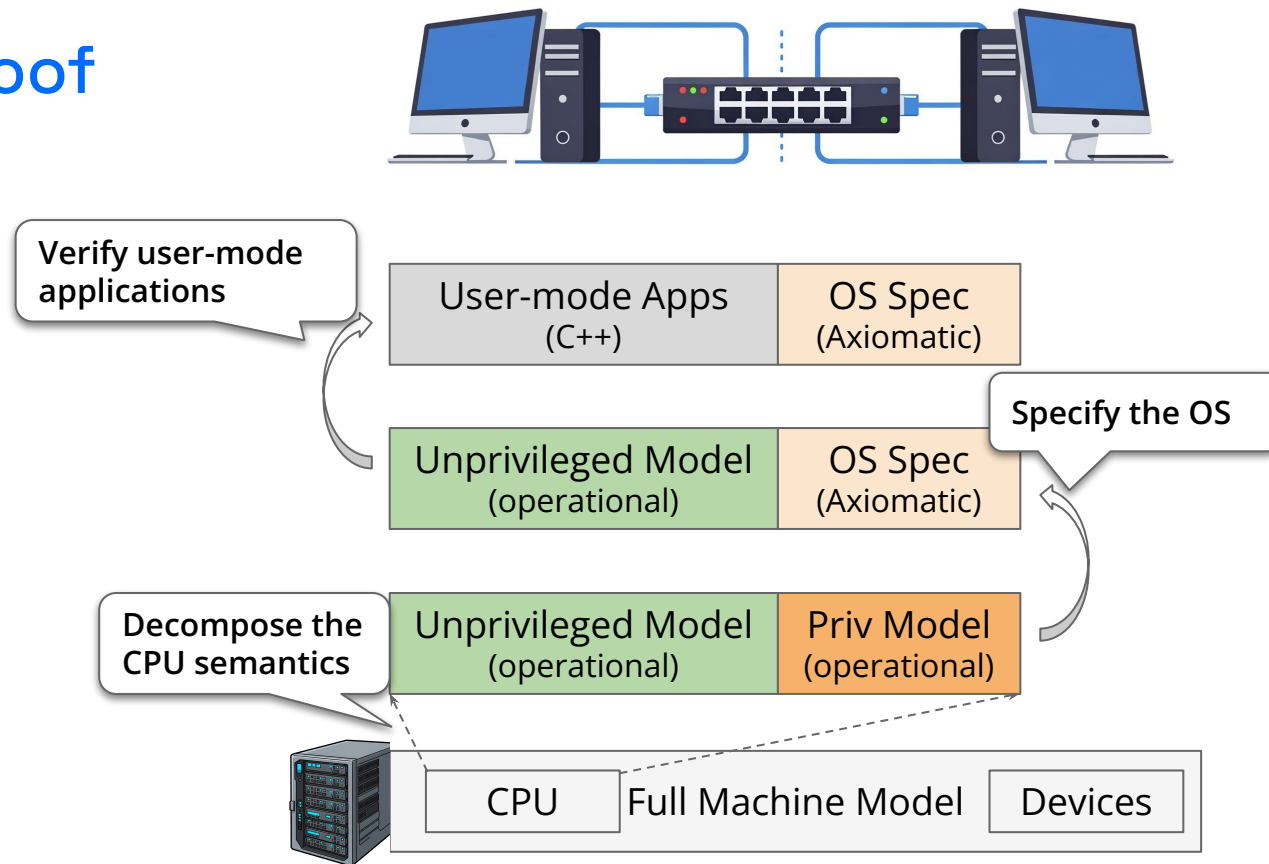
The Full-Stack Proof



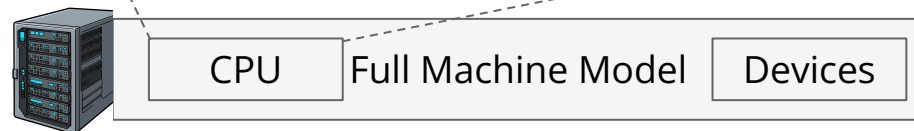
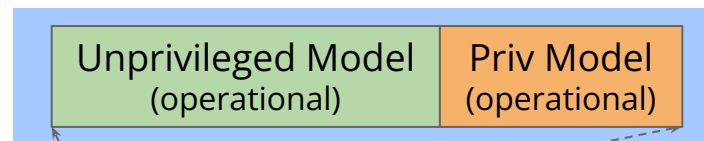
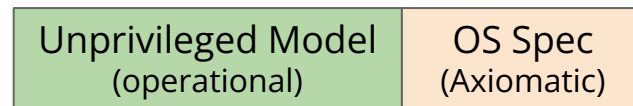
The Full-Stack Proof



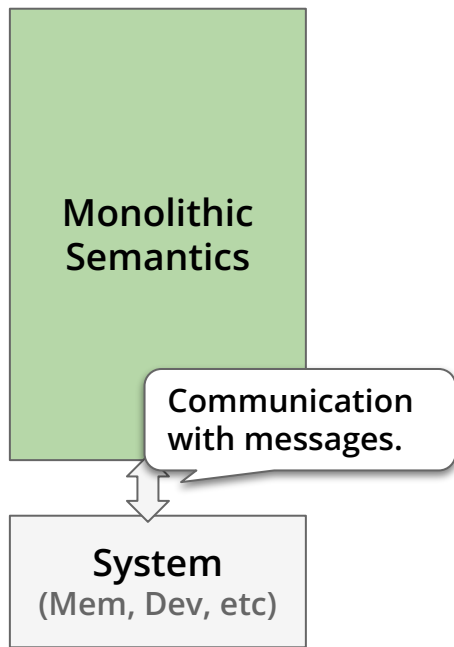
The Full-Stack Proof



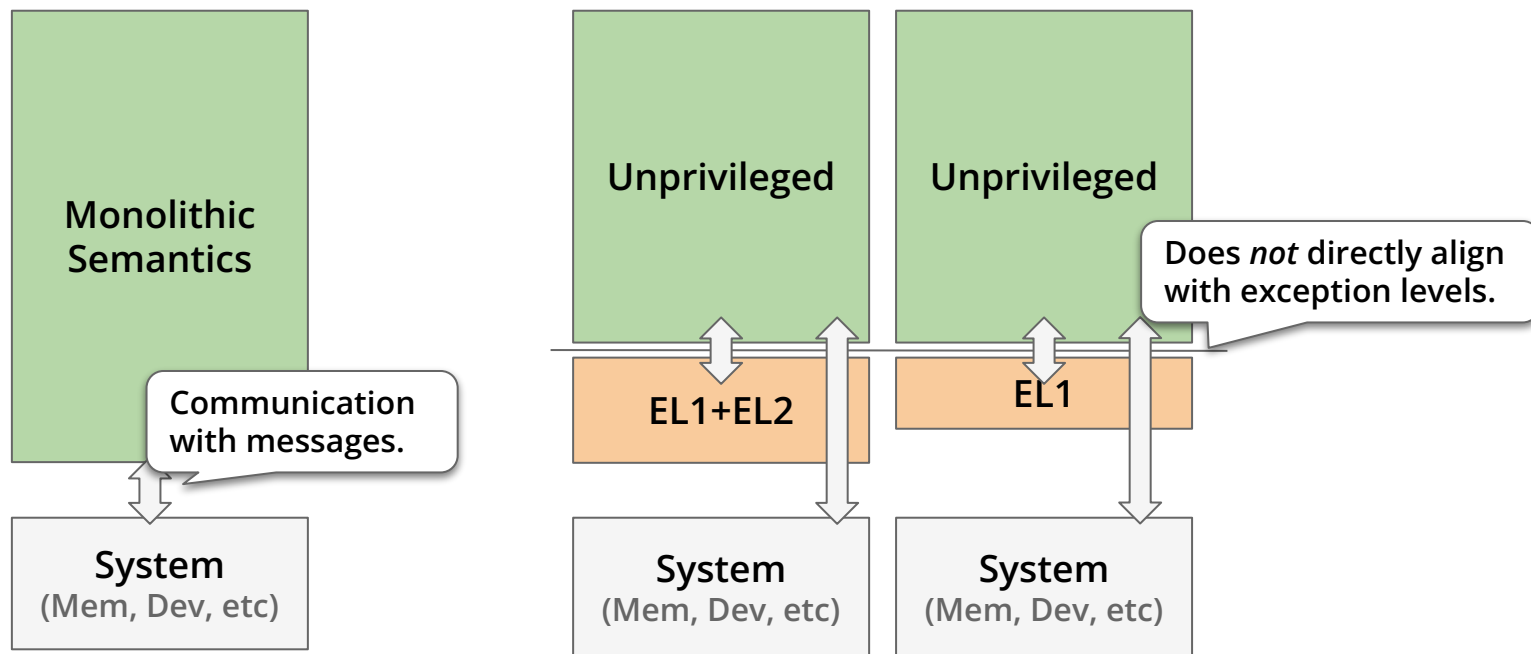
The Full-Stack Proof



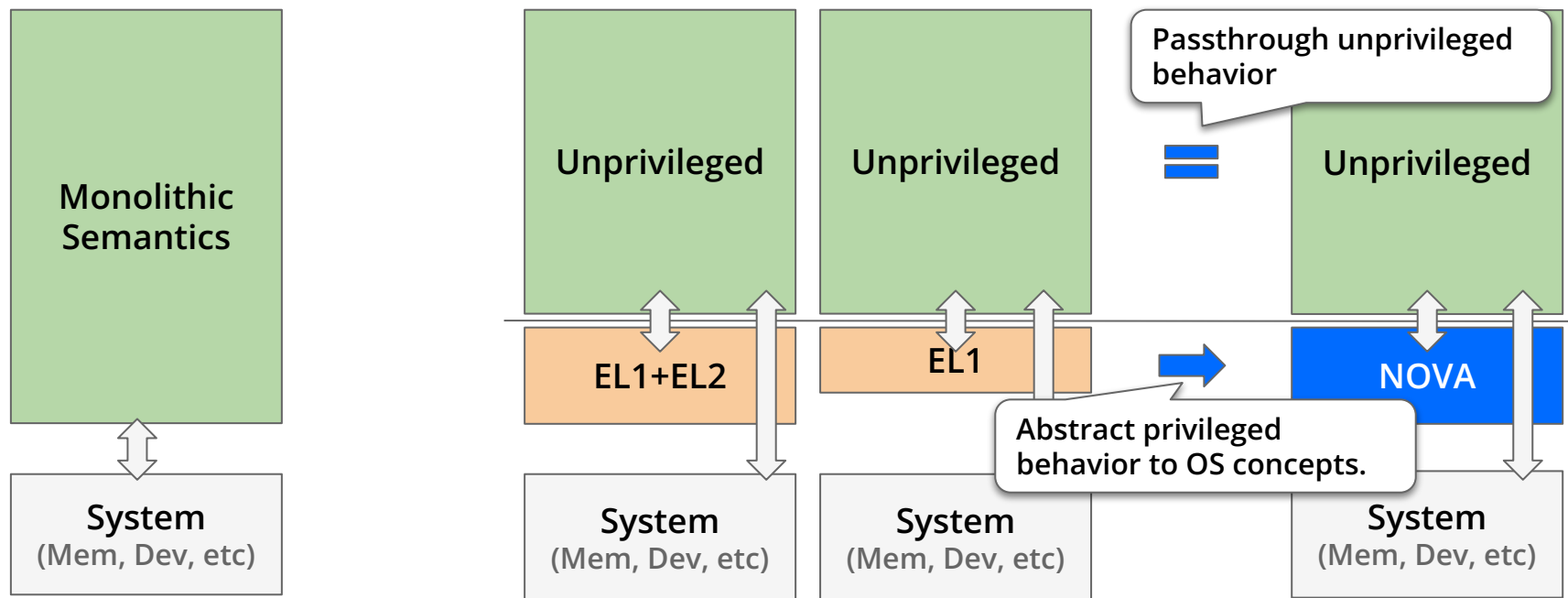
Decomposing CPU Semantics



Decomposing CPU Semantics

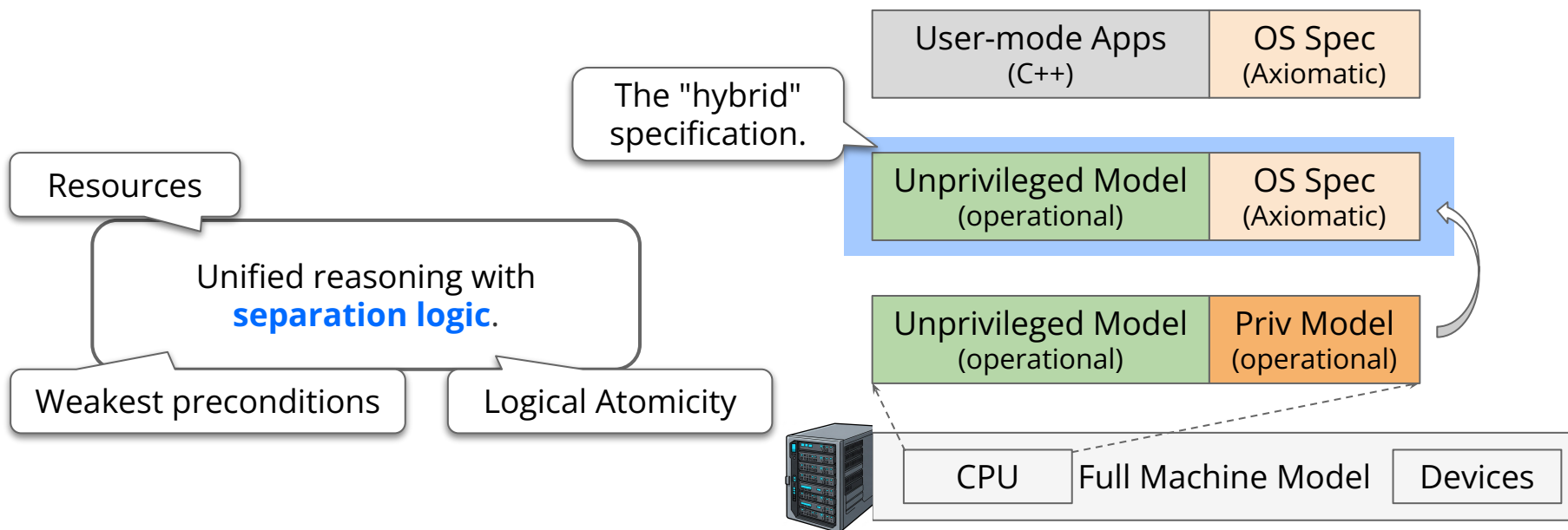


Decomposing CPU Semantics



The Full-Stack Proof

Specify the OS



The NOVA Machine

This is a "thread"-local reasoning principle!

Theorem $\text{wp_nova_ec_intro} : \forall \text{ ec regs},$
 $(\forall \text{ evt regs'},$

$[\mid \text{cpu.step regs evt regs'} \mid] - *$

match evt with

$\mid \text{None} \Rightarrow \text{wp_nova_ec regs'}$

$\mid \text{Some syscall} \Rightarrow \text{wp_hypercall ec ..}$

$\mid \text{Some (mem vaddr)} \Rightarrow \text{wp_mem vaddr}$

$\mid \dots$

$\text{end})$

$\vdash \text{wp_nova_ec ec regs}.$

Hypercall behavior

Address translation specified using NOVA state.

Unprivileged

NOVA

System

(Mem, Dev, etc)

NOVA's Capability API:

kernel objects, hypercalls, and HW-assisted virtualization

PD: protection domain with capabilities in Object spaces

EC: execution contexts
SC: scheduling contexts
PT: portals

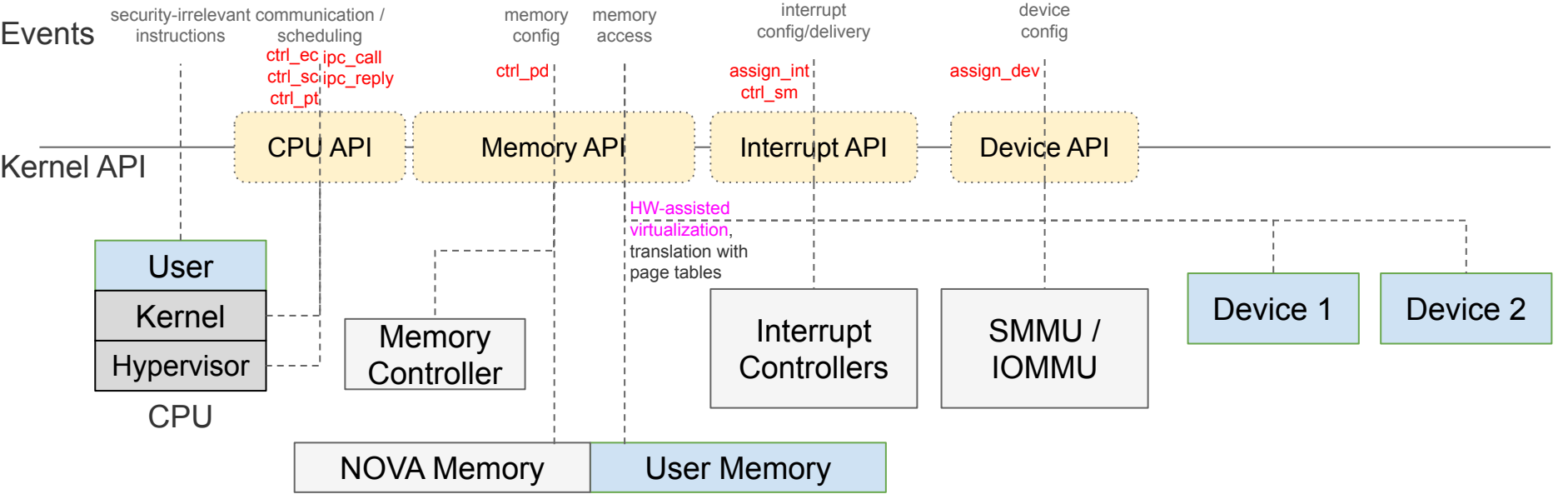
Memory spaces

SM: semaphores

DMA spaces

user-owned

μkernel-owned



NOVA's Capability API:

kernel objects, hypercalls, and HW-assisted virtualization

PD: protection domain with capabilities in Object spaces

EC: execution contexts
SC: scheduling contexts
PT: portals

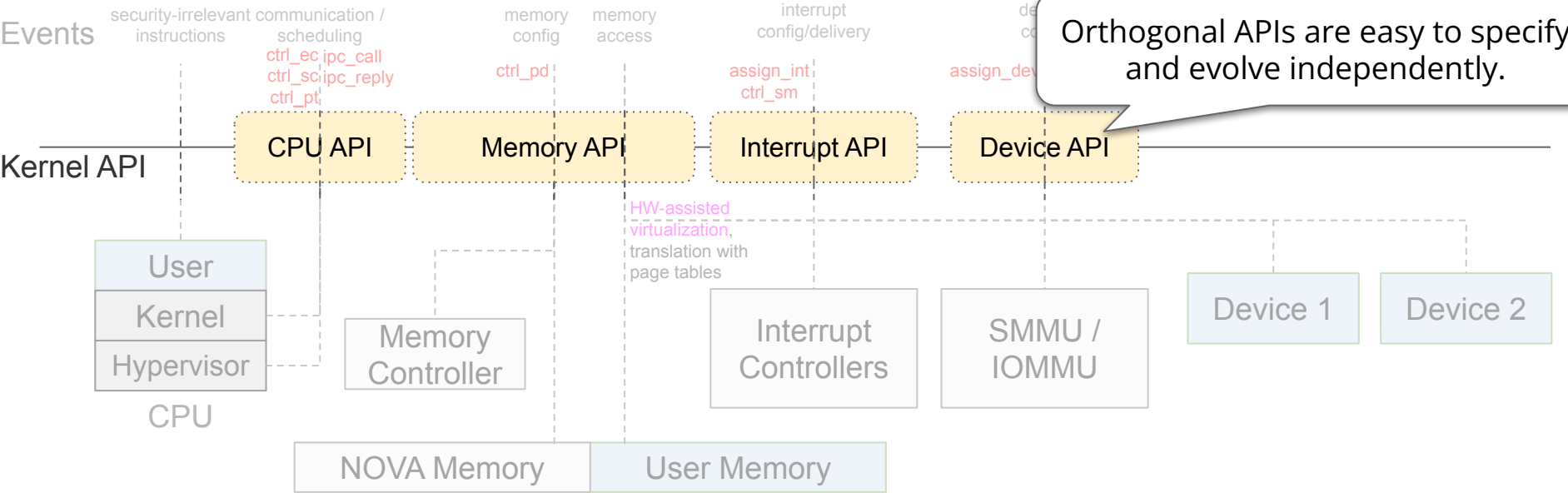
Memory spaces

SM: semaphores

DMA spaces

user-owned

μkernel-owned



NOVA's Capability API:

kernel objects, hypercalls, and HW-assisted virtualization

PD: protection domain with capabilities in Object spaces

EC: execution contexts
SC: scheduling contexts
PT: portals

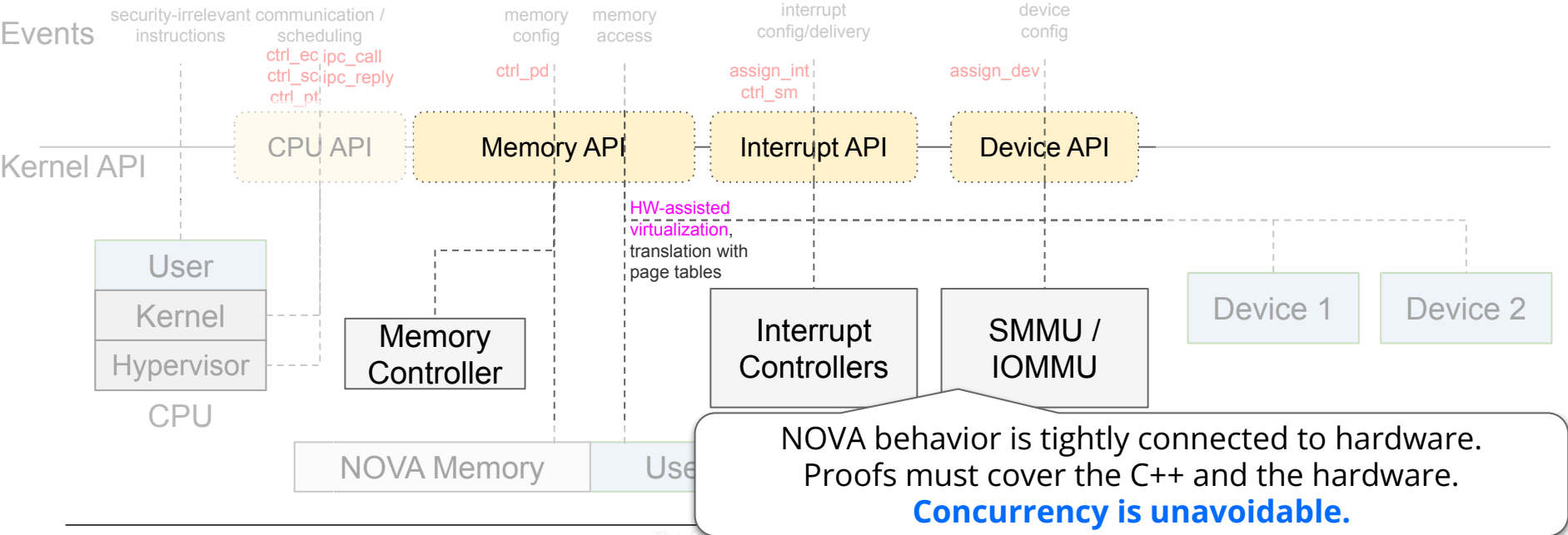
Memory spaces

SM: semaphores

DMA spaces

user-owned

μkernel-owned



Fair Semaphores

4.4.5 Control Semaphore

Parameters:

```
status = ctrl_sm (SEL_obj sm,           // Semaphore
                  UINT ticks);          // Deadline Timeout
```

Flags:

0	0	Z	D
3	2	1	0

Description:

Prior to the hypercall:

- If **D=0 (Up)**: { $PD_{CURRENT}$, SEL_{OBJ} sm } must refer to a **SM Object Capability** ($CAP_{OBJ_{SE}}$) with permission $CTRL_{UP}$.
- If **D=1 (Down)**: { $PD_{CURRENT}$, SEL_{OBJ} sm } must refer to a **SM Object Capability** ($CAP_{OBJ_{SE}}$) with permission $CTRL_{DN}$.

If the hypercall completed successfully:

- If **D=0 (Up)**: if there were **ECs** blocked on the semaphore, then the microhypervisor has released one of those blocked **ECs**. Otherwise, the microhypervisor has incremented the semaphore counter. The deadline timeout value and the Z-flag were ignored.
- If **D=1 (Down)**: if the semaphore counter was larger than zero, then the microhypervisor has decremented the semaphore counter (**Z=0**) or set it to zero (**Z=1**). Otherwise, the microhypervisor has blocked $EC_{CURRENT}$ on the semaphore. If the deadline timeout value was non-zero, $EC_{CURRENT}$ unblocks with a timeout status when the architectural timer reaches or exceeds the specified ticks value.

Blocking and releasing of **ECs** on a semaphore uses the FIFO queuing discipline.

Status:

SUCCESS

- The hypercall completed successfully.

TIMEOUT

- If **D=1**: Down operation aborted when the timeout triggered.

OVRFLOW

- If **D=0**: Up operation aborted because the semaphore counter would overflow.

BAD_CAP

- { $PD_{CURRENT}$, SEL_{OBJ} sm } did not refer to a **SM Object Capability** ($CAP_{OBJ_{SE}}$) or that capability had insufficient permissions.

BAD_CPU

- If **D=1** on an interrupt semaphore: Attempt to wait for the interrupt on a different **CPU** than the **CPU** to which that interrupt has been routed via [assign_int](#).

Fair Semaphores State

A minimal piece of independent state and an ownership discipline/"protocol"

Reflected as **separation logic resources**

- Selectors map to capabilities

$\text{pd};\text{sel} \mapsto (\text{obj}, \text{perm})$

- The counter of a semaphore

$\text{sm.counter obj } n$

- The queue of blocked threads

$\text{sm.queue obj } ls$

4.4.5 Control Semaphore

Parameters:

```
status = ctrl_sm (SEL_obj sm,           // Semaphore
                  UINT ticks);          // Deadline Timeout
```

Prior to the hypercall:

- If $D=0$ (Up): $\{ \text{PD}_{\text{CURRENT}}, \text{SEL}_{\text{OBJ}} \text{ sm} \}$ must refer to a SM Object Capability ($\text{CAP}_{\text{OBJ}_{\text{SM}}}$) with permission CTRL_{UP} .
- If $D=1$ (Down): $\{ \text{PD}_{\text{CURRENT}}, \text{SEL}_{\text{OBJ}} \text{ sm} \}$ must refer to a SM Object Capability ($\text{CAP}_{\text{OBJ}_{\text{SM}}}$) with permission CTRL_{DN} .

If the hypercall completed successfully:

- If $D=0$ (Up): if there were ECs blocked on the semaphore, then the microhypervisor has released one of those blocked ECs . Otherwise, the microhypervisor has incremented the semaphore counter. The deadline timeout value and the Z-flag were ignored.
- If $D=1$ (Down): if the semaphore counter was larger than zero, then the microhypervisor has decremented the semaphore counter ($Z=0$) or set it to zero ($Z=1$). Otherwise, the microhypervisor has blocked $\text{EC}_{\text{CURRENT}}$ on the semaphore. If the deadline timeout value was non-zero, $\text{EC}_{\text{CURRENT}}$ unblocks with a timeout status when the architectural timer reaches or exceeds the specified ticks value.

Blocking and releasing of ECs on a semaphore uses the FIFO queuing discipline.

Status:

SUCCESS

- The hypercall completed successfully.

TIMEOUT

- If $D=1$: Down operation aborted when the timeout triggered.

OVRFLOW

- If $D=0$: Up operation aborted because the semaphore counter would overflow.

BAD_CAP

- $\{ \text{PD}_{\text{CURRENT}}, \text{SEL}_{\text{OBJ}} \text{ sm} \}$ did not refer to a SM Object Capability ($\text{CAP}_{\text{OBJ}_{\text{SM}}}$) or that capability had insufficient permissions.

BAD_CPU

- If $D=1$ on an interrupt semaphore: Attempt to wait for the interrupt on a different CPU than the CPU to which that interrupt has been routed via assign_int.

Fair Semaphores Behavior

Reflected as **separation logic resources**

- Selectors map to capabilities

`pd;sel ↦ (obj, perm)`

- The counter of a semaphore

`sm.counter obj n`

- The queue of blocked threads

`sm.queue obj ls`

4.4.5 Control Semaphore

Parameters:

```
status = ctrl_sm (SEL_obj sm,           // Semaphore
                  UINT ticks);          // Deadline Timeout
```

Flags:

0	0	Z	D
3	2	1	0

Description:

Prior to the hypercall:

- If **D=0 (Up)**: { `PDCURRENT`, `SELobj sm` } must refer to a SM Object Capability (`CAPobj,se`) with permission `CTRLup`.
- If **D=1 (Down)**: { `PDCURRENT`, `SELobj sm` } must refer to a SM Object Capability (`CAPobj,se`) with permission `CTRLdn`.

If the hypercall completed successfully:

- If **D=0 (Up)**: if there were ECs blocked on the semaphore, then the microhypervisor has released one of those blocked ECs. Otherwise, the microhypervisor has incremented the semaphore counter. The deadline timeout value and the Z-flag were ignored.
- If **D=1 (Down)**: if the semaphore counter was larger than zero, then the microhypervisor has

Sequencing

```
let (Some obj) :=
  resolve_sm ec.pd {UP} sel else K BAD_CAP;
sm.incr obj K
⊢ wp_hypercall ec ctrl_sm(0, sel, _) K
```

Continuation.

• If **D=1**: Down operation aborted when the time out occurred. **OVRFLOW**

• If **D=0**: Up operation aborted because the semaphore counter would overflow. **BAD_CAP**

BAD_CAP

• { `PDCURRENT`, `SELobj sm` } did not refer to a SM Object Capability (`CAPobj,se`) or that capability had insufficient permissions. **BAD_CPU**

BAD_CPU

• If **D=1** on an interrupt semaphore: Attempt to wait for the interrupt on a different **CPU** than the **CPU** to which that interrupt has been routed via `assign_int`.

Parameters:

```
status = ctrl_sm (SEL_obj sm,           // Semaphore
                  UINT ticks);         // Deadline Timeout
```

Fair Semaphores

Behavior

Reflected

$$sm.incr\ g\ K :=$$

$$AU \ll \exists n. sm.counter\ g\ n \gg @N$$

$$\ll sm.counter\ g\ (cap\ (n+1)),$$

$$COMM\ K\ (if\ overflows\ (n+1)$$

$$then\ OVERFLOW\ else\ SUCCESS) \gg$$

- Select

pd

- The

$$sm.counter\ obj\ n$$

- The queue of blocked threads

$$sm.queue\ obj\ ls$$

Exchange this...

...for this

Afterwards, continue

```

    resolve_sm ec.pd {UP} sel else K BAD_CAP;
    sm.incr obj K
  ⊢ wp_hypercall ec ctrl_sm(0, sel, _) K

```

- If D=1: Down operation aborted when the timeout triggered.

OVERFLOW

- If D=0: Up operation aborted because the semaphore counter would overflow.

BAD_CAP

- { $PDCURRENT$, $SEL_obj\ sm$ } did not refer to a SM Object Capability (CAP_obj_se) or that capability had insufficient permissions.

BAD_CPU

- If D=1 on an interrupt semaphore: Attempt to wait for the interrupt on a different CPU than the CPU to which that interrupt has been routed via [assign_int](#).

Fair Semaphores Behavior

Reflected as **separation logic resource**

- Selectors map to capabilities

$pd; sel \mapsto (obj, perm)$

- The counter of a semaphore

$sm.counter\ obj\ n$

Two atomic steps.

- The queue of blocked threads

$sm.queue\ obj\ ls$

4.4.5 Control Semaphore

Parameters:

```
status = ctrl_sm (SEL_obj sm,           // Semaphore
                  UINT ticks);          // Deadline Timeout
```

Flags:

0	0	Z	D
3	2	1	0

Description:

```
resolve_sm pd sel K :=
  AU<<∃ obj,perm. pd; sel↦(obj,perm)>> @NOVA
  <<ec.pd; sel↦(obj,perm),
    COMM K (if {UP} ⊆ perm
            then Some obj else None)>>
```

```
let (Some obj) :=
  resolve_sm ec.pd {UP} sel else K BAD_CAP;
sm.incr obj K
```

```
sm.incr g K :=
  AU<<∃ n. sm.counter g n>> @NOVA
  <<sm.counter g (cap (n+1)),
    COMM K (if overflows (n+1)
            then OVERFLOW else SUCCESS)>>
```

- If D=1 on an interrupt semaphore: Attempt to wait for the interrupt on a different CPU than the CPU to which that interrupt has been routed via [assign_int](#).

Fair Semaphores Behavior

Reflected as **separation logic resource**

- Selectors map to capacity

`pd;sel ↦ (obj, pe`

- The counter of

`sm.counter c`

- The queue of blocked

`sm.queue obj ls`

Verified
ROCC

1. Connect the abstract state to C++ state
2. Apply the linearization points when updating the state.

```
AU<<∃ n. sm.counter g n>> @NOVA
<<sm.counter g (cap (n+1)),
  COMM K (if overflows (n+1)
    then OVERFLOW else SUCCESS)>>
```

4.4.5 Control Semaphore

Parameters:

```
status = ctrl_sm (SEL_obj sm,           // Semaphore
                  UINT ticks);          // Deadline Timeout
```

Flags:

0 0 7 D

>> @NOVA

then Some obj

(UP) sel_obj K PAD_CAP;

- If D=1 on an interrupt semaphore: Attempt to wait for the interrupt on a different CPU than the CPU to which that interrupt has been routed via `assign_int`.

Spawning Threads

Concurrent Behavior

Thread creation occurs when scheduling contexts are bound to global execution contexts.

The proof obligation for a thread creation is expressed as a WP for the new thread.

```

let g_pd := resolve_pd ec.pd sel_pd {SC} Q in
let g_ec := resolve_ec ec.pd sel_ec {EC_BIND_SC} Q in
let g_sc := sc.create ec.pd sel_sc Q in
let run := ec.bind g_ec g_sc in

(if run then wp_nova_ec g_ec else emp)* K SUCCESS
⊢ wp_hypercall ec create_sc(sel_sc, sel_pd, sel_ec, scd) K

```

5.3.3 Create Scheduling Context

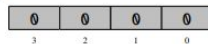
Parameters:

```

status = create_sc (SEL_OBJ sel,      // Created SC
                   SEL_OBJ pd,       // Owner PD
                   SEL_OBJ ec,       // Bound EC
                   SCD scd)          // Scheduling Context Descriptor

```

Flags:



Description:

Creates a new [Scheduling Context \(SC\)](#).

Prior to the hypercall:

- $SPC_{OBJ_CURRENT}[sel]$ must refer to a [Null Capability \(CAP₀\)](#).
- $SPC_{OBJ_CURRENT}[pd]$ must refer to a [PD Capability \(CAP_{OBJ_{PD}}\)](#) with permission SC.
- $SPC_{OBJ_CURRENT}[ec]$ must refer to an [EC Capability \(CAP_{OBJ_{EC}}\)](#) with permission BIND_{SC}.

If the hypercall completed successfully:

- A new [Scheduling Context \(SC\)](#) has been created.
- The created [SC](#) is bound to the [EC](#) referred to by $SPC_{OBJ_CURRENT}[ec]$ on the [CPU](#) of that [EC](#), with its scheduling parameters set according to scd .
- The resources for the created [SC](#) were accounted to the [PD](#) referred to by $SPC_{OBJ_CURRENT}[pd]$.

BAD_PAR

- At least one [SCD](#) field in scd was invalid.

MEM_OBJ

- The [Protection Domain](#) referred to by $SPC_{OBJ_CURRENT}[pd]$ had insufficient memory resources for

Spawning Threads

Concurrent Behavior

Thread creation occurs when scheduling contexts are bound to global execution contexts.

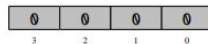
The proof obligation for a thread creation is expressed as a WP for the new thread.

5.3.3 Create Scheduling Context

Parameters:

```
status = create_sc (SEL_OBJ sel,      // Created SC
                  SEL_OBJ pd,        // Owner PD
                  SEL_OBJ ec,        // Bound EC
                  SCD scd)           // Scheduling Context Descriptor
```

Flags:



Description:

Creates a new [Scheduling Context \(SC\)](#).

Prior to the hypercall:

- $SPC_{OBJ_CURRENT}[sel]$ must refer to a Null Capability (CAP_0).
- $SPC_{OBJ_CURRENT}[pd]$ must refer to a PD Capability (CAP_{OBJ_PD}) with permission SC.
- $SPC_{OBJ_CURRENT}[ec]$ must refer to an EC Capability (CAP_{OBJ_EC}) with permission $BIND_{sc}$.

If the hypercall completed successfully:

- A new [Scheduling Context \(SC\)](#) has been created.
- The created [SC](#) is bound to the [EC](#) referred to by $SPC_{OBJ_CURRENT}[ec]$ on the [CPU](#) of that [EC](#), with its scheduling parameters set according to scd .
- The resources for the created [SC](#) were accounted to the [PD](#) referred to by $SPC_{OBJ_CURRENT}[pd]$.

Prove an extra WP for the new thread.

Continue running the current thread.

```
(if run then wp_nova_ec g_ec else emp) * K SUCCESS
⊢ wp_hypercall ec create_sc(sel_sc, sel_pd, sel_ec, scd) K
```

BAD_PAR

- At least one [SCD](#) field in scd was invalid.

MEM_OBJ

- The [Protection Domain](#) referred to by $SPC_{OBJ_CURRENT}[pd]$ had insufficient memory resources for

Spawning Threads

Concurrent Behavior

Thread creation occurs when scheduling contexts are bound to global execution contexts.

The proof obligation for a thread creation is expressed as a WP for the new thread.

Prove an extra WP for the new thread.

Need logical atomicity to make this complete.

Independence is the core of concurrency!

Continue running the current thread.

$$\begin{aligned} & (\text{if run then } \underline{\text{wp_nova_ec } g_ec} \text{ else emp}) * K \underline{\text{SUCCESS}} \\ \vdash & \text{wp_hypercall ec create_sc(sel_sc, sel_pd, sel_ec, scd) K} \end{aligned}$$

5.3.3 Create Scheduling Context

Parameters:

```
status = create_sc (SEL_OBJ sel,      // Created SC
                   SEL_OBJ pd,      // Owner PD
                   SEL_OBJ ec,      // Bound EC
                   SCD scd)         // Scheduling Context Descriptor
```

Flags:

0	0	0	0
3	2	1	0

Description:

Creates a new [Scheduling Context \(SC\)](#).

Prior to the hypercall:

- $\text{SPC}_{\text{OBJ_CURRENT}}[\text{sel}]$ must refer to a Null Capability (CAP_0).
- $\text{SPC}_{\text{OBJ_CURRENT}}[\text{pd}]$ must refer to a PD Capability ($\text{CAP}_{\text{OBJ}_{\text{PD}}}$) with permission SC.
- $\text{SPC}_{\text{OBJ_CURRENT}}[\text{ec}]$ must refer to an EC Capability ($\text{CAP}_{\text{OBJ}_{\text{EC}}}$) with permission BIND_{SC} .

If the hypercall completed successfully:

- A new [Scheduling Context \(SC\)](#) has been created.
- The created SC is bound to the EC referred to by $\text{SPC}_{\text{OBJ_CURRENT}}[\text{ec}]$ on the CPU of that EC, with its PD referred to by $\text{SPC}_{\text{OBJ_CURRENT}}[\text{pd}]$.

BAD_PAR

- At least one SCD field in scd was invalid.

MEM_OBJ

- The [Protection Domain](#) referred to by $\text{SPC}_{\text{OBJ_CURRENT}}[\text{pd}]$ had insufficient memory resources for

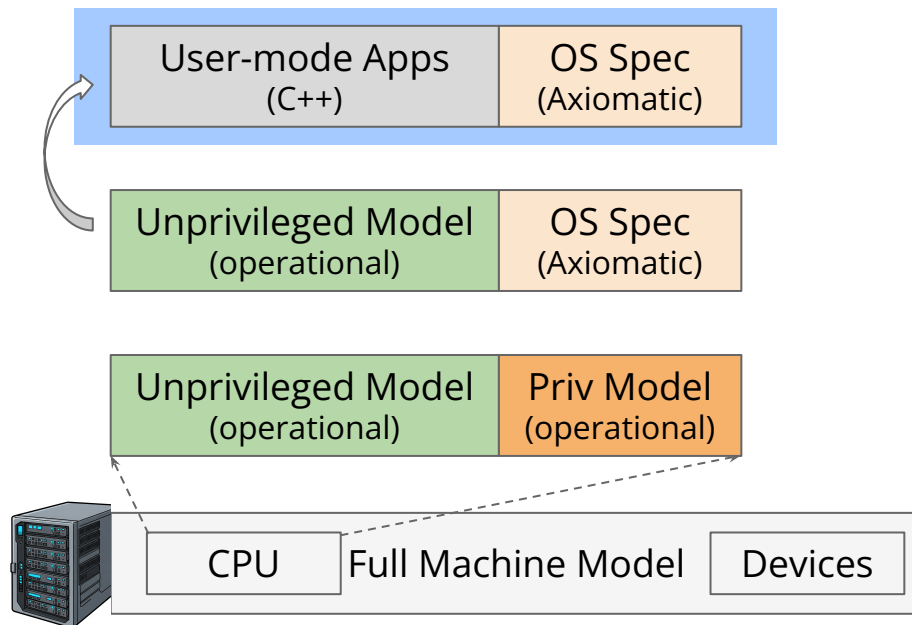
The Full-Stack Proof

User-mode Verification

- Device drivers.
 - PL011
 - (Simple-mode) Zynqmp
- Terminal multiplexor
- VIRTIO-based layer 2 switch.
- Virtual Machine Monitor

Unified reasoning with
separation logic.

Highly reusable proofs



Uniform Reasoning in Separation Logic

Definition `portal_spec_cpu`

```
(exit_reason : evt.t nova.g
(yg : GlobalRoundupInfo.ghos
(cy : bm.cpu.Name → Vcpu.gna
(shareds : bm.cpu.Name → Vcpu.shared_t)
(bm_cid : bm.cpu.Name)
: WpSpec_cpp :=
(** Sequential ownership of current VCPU -- C++/NOVA *)
\let yvcpu := cy bm_cid
\pre{mseq : Vcpu.seq_t} vcpu |-> Vcpu.seqR (cQp.m 1) bm_cid yvcpu mseq

(** Shared handle to the <<Board>> -- C++/NOVA *)
\let pd := (Model.Cpu.pd_name (Vcpu.base_basey yvcpu))
\prepost{yboard qboard mboard} Model.Board.P pd qboard yboard mboard
\require yboard .^ Model.BoardImpl.Model._stage = Model.Board.SteadyState

(** Shared handles to other VCPUs -- C++/NOVA *)
\prepost{q_vcpus} _global "Model::vcpus" |-> Vcpu.vcpusR cy shareds q_vcpus yg

(** VCPU registers -- NOVA *)
\prepost{vcpu_regs : arch.regs_t guest}
  Model.Cpu.ecRegs (Vcpu.base_basey yvcpu) vcpu_regs

(** Specification state of the Guest -- spec *)
\pre{bm_state : lts_state (bm.cpu.lts _)} bm.cpu.core bm_cid bm_state
\require{decode_assist} cpu.pendingInstruction bm_state decode_assist
\require{fault_regs} VCPU.FaultRegs.reg_accessor_encodes_decodeassist fault_regs
  exit_reason decode_assist

(** Spec & Implementation state are related! *)
\require CPU.related_full_states bm_state vcpu_regs

(* ... other ownership ... *)
```

User-mode entry point (from hardware virtualization)

Uniform Reasoning in Separation Logic

C++ state.

and some
NOVA state

NOVA State

Specification state

Simulation

```
Definition portal_spec_cpu
  (exit_reason : evt.t nova.g
  (yg : GlobalRoundupInfo.ghos
  (cy : bm.cpu.Name → Vcpu.gna
  (shares : bm.cpu.Name → Vcpu.shared_t)
  (bm_cid : bm.cpu.Name)
  : WpSpec_cpp :=

  (** Sequential ownership of current VCPU -- C++/NOVA *)
  \let yvcpu := cy bm_cid
  \pre{mseq : Vcpu.seq_t} vcpu |-> Vcpu.seqR (cQp.m 1) bm_cid yvcpu mseq

  (* Shared handle to the <<Board>> -- C++/NOVA *)
  \let pd := (Model.Cpu.pd_name (Vcpu.base_basey yvcpu))
  \prepost{yboard qboard mboard} Model.Board.P pd qboard yboard mboard
  \require yboard .^ Model.BoardImpl.Model._stage = Model.Board.SteadyState

  (** Shared handles to other VCPUs -- C++/NOVA *)
  \prepost{q_vcpus} _global "Model::vcpus" |-> Vcpu.vcpusR cy shares q_vcpus yg

  (** VCPU registers -- NOVA *)
  \prepost{vcpu_regs : arch.regs_t guest}
  Model.Cpu.ecRegs (Vcpu.base_basey yvcpu) vcpu_regs

  (** Specification state of the Guest -- spec *)
  \pre{bm_state : lts_state (bm.cpu.lts _)} bm.cpu.core bm_cid bm_state
  \require{decode_assist} cpu.pendingInstruction bm_state decode_assist
  \require{fault_regs} VCPU.FaultRegs.reg_accessor_encodes_decodeassist fault_regs
  exit reason decode_assist

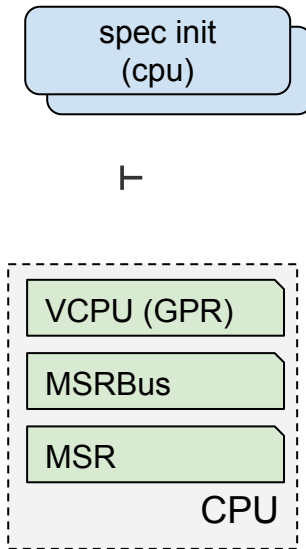
  (** Spec & Implementation state are related! *)
  \require CPU.related_full_states bm_state vcpu_regs

  (* ... other ownership ... *)
```

User-mode entry point (from
hardware virtualization)

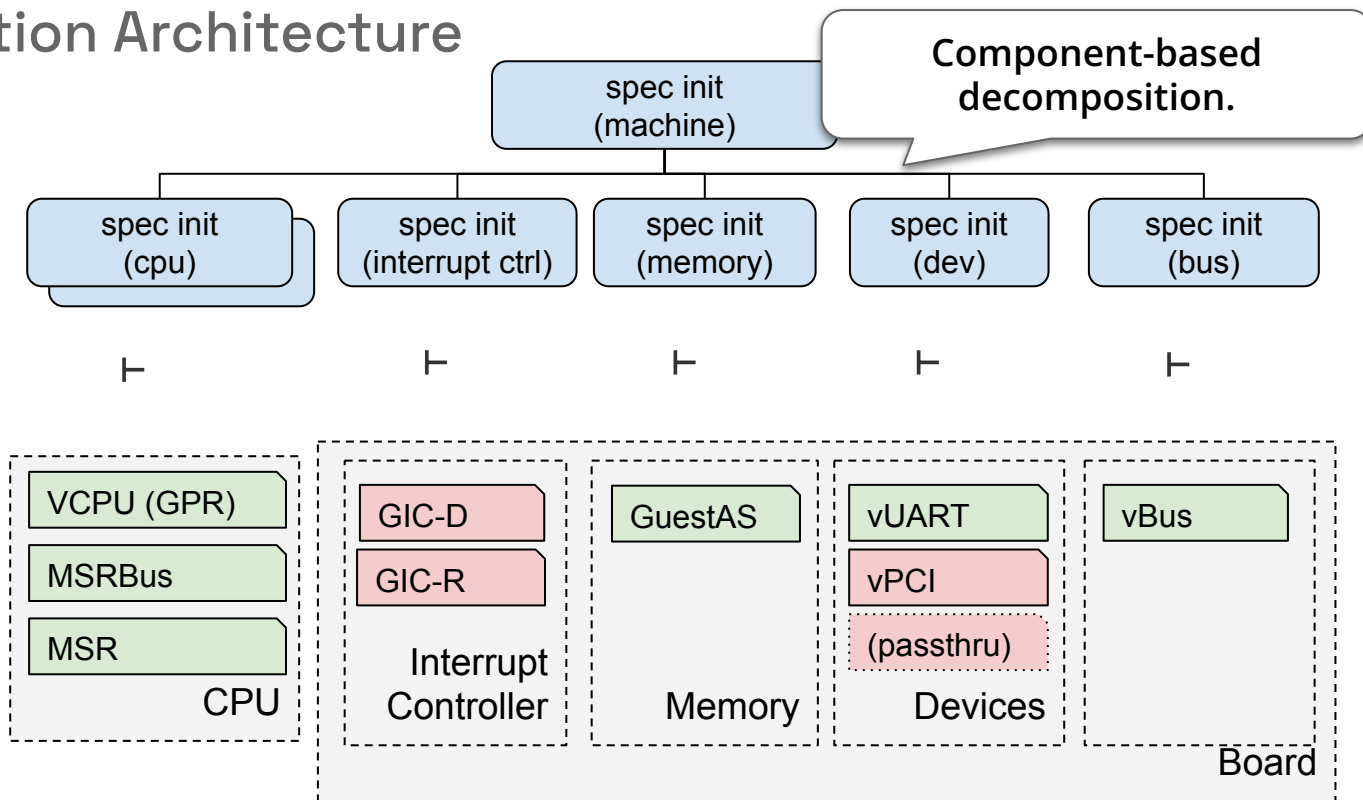
Virtual Machine Monitor

Verification Architecture



Virtual Machine Monitor

Verification Architecture



Takeaways

- Separation logic is a powerful formalism for reasoning about dynamic systems.
- It can be used to specify complex, concurrent behavior.
- It can be used to *modularly verify* complex, concurrent behavior.
- It works for the (good) *code that you have right now*.

The Future

- Specify your next system using SL!
- Verify the implementation!

Future Research

- Refine low-level hardware models
 - Security reasoning.
 - Weak memory.
- Liveness verification.
 - Availability.
- Hyper-properties.
 - Confidentiality.
 - Isolation.